

Paper Empire

Technische Architektur und Techplan

Desktop/Steam-first - Godot 4 - Codex-taugliche Fassung

Stand: 2026-05-09

Entscheidung

Paper Empire wird ab jetzt nicht mehr als Webspiel geplant. Ziel ist ein Desktop-PC-Spiel mit Steam als späterer Hauptdistribution. Die technische Basis ist Godot 4 mit GDScript. Web, Mobile und Electron werden aus dem Hauptplan entfernt.

0. Zweck dieses Dokuments

Dieses Dokument ersetzt den vorherigen Web/Electron-Techplan. Es beschreibt die neue Zielarchitektur fuer Paper Empire als Desktop/Steam-first-Spiel mit Godot 4. Es ist bewusst so formuliert, dass Codex spaeter daraus konkrete Tasks ableiten kann, ohne die gesamte Architektur neu erfinden zu muessen.

Der separate detaillierte Schrittplan wird spaeter erstellt. Dieses Dokument legt nur Architektur, Techstack, Regeln, Verantwortlichkeiten, Datenfluesse und technische Leitplanken fest.

1. Executive Summary

Bereich	Festlegung
Zielplattform	Windows-PC zuerst; Steam spaeter als Hauptdistribution; Linux/macOS optional nach dem Windows-MVP.
Engine	Godot 4.x, Desktop-Export, 2D-Projekt.
Sprache	GDScript. Kein C# im Hauptplan.
Steam	Spaeter ueber GodotSteam oder passende Steamworks-Integration. Nicht im ersten Prototyp hardcoden.
Karte	2D-Weltkarte mit Laendern als eigene klickbare Areas/Polygone.
Simulation	Eigene Game-Simulation mit festem Simulationstick und getrennten visuellen Animationen.
UI	Godot Control Nodes fuer HUD, Panels, Modals, Shop, Army Editor.
Effekte	Godot Tweens, AnimationPlayer, GPUParticles2D/CPUParticles2D, AudioStreamPlayer.
Save	Lokale Save-Dateien unter user:// mit Save-Versionierung; Steam Cloud spaeter.
Codex	VSCoDe + Codex. Codex arbeitet in kleinen Tasks, vor allem an GDScript, Daten und klar begrenzten Szenen.

2. Warum die Entscheidung auf Godot 4 faellt

Die vorherige Electron/React-Architektur war sinnvoll, solange eine Webversion ein echtes Ziel war. Da Web jetzt ausdruecklich aus dem Plan entfernt wurde und Desktop/Steam das Hauptziel ist, verschiebt sich die Abwaegung klar Richtung Godot.

- Godot bringt eine echte Game-Loop-Struktur mit `_process()` und `_physics_process()`. Dadurch lassen sich visuelle Updates und feste Simulationsschritte sauber trennen.
- 2D-Kamera, Zoom, Pan, Animationen, Tweens, Partikel und Audio sind direkt in der Engine vorhanden.
- Godot exportiert direkt zu Desktop-Plattformen. Kein Electron-Wrapper, keine WebView-Abhaengigkeiten und keine native Node-Modul-Probleme.
- Steamworks-Integration ist ueber GodotSteam naeher am klassischen Game-Engine-Workflow als bei Electron.
- Die spaeter geplanten symbolischen Raketen, Explosionen, Armee-Bewegungen, Sounds und Kartenanimationen passen sehr gut in Godots 2D-System.

Wichtige Korrektur gegeneuber dem alten Plan

Das Problem war nicht, dass Web-Technologie Ticks oder Echtzeit-Armeebewegungen nicht kann. Das kann sie. Das eigentliche Argument fuer Godot ist: Ohne Webziel muss Paper Empire nicht mehr wie eine Webapp gebaut werden. Godot reduziert den selbstgebauten Unterbau fuer Desktop, Kamera, Effekte, Audio, Packaging und Steam-nahe Features.

3. Nicht-Ziele

- Keine Webversion im Hauptplan.
- Keine Mobile-Version im MVP.
- Keine Electron-, React-, Vite- oder Browser-Architektur.
- Kein Multiplayer.
- Keine Serverpflicht.
- Keine Steamworks-Features im ersten Prototyp.
- Keine komplette Weltkarte mit 180+ echten Laenderformen im allerersten technischen Prototyp.
- Kein komplexes ECS-System, solange Godot-Nodes und klare Manager reichen.

4. Techstack

Bereich	Festlegung
Engine	Godot 4.x stable. Fuer den Start eine konkrete Version festpinnen, z. B. Godot 4.4 oder neuer, sobald installiert.
Rendering	Godot 2D. Laender ueber Polygon2D/Area2D/CollisionPolygon2D oder importierte Map-Shapes.
Scripting	GDScript mit statischer Typisierung, class_name, klaren Interfaces und kleinen Dateien.
UI	Godot Control Nodes: Panels, Containers, Buttons, Modals, Cards, Tooltips.
Animationen	Tween, AnimationPlayer, AnimationTree nur falls noetig.
Partikel	GPUParticles2D fuer Effekte; CPUParticles2D falls GPU-Partikel fuer bestimmte Effekte unpraktisch sind.
Audio	AudioStreamPlayer, AudioStreamPlayer2D, Audio-Busse fuer Master/Music/SFX/UI.
Daten	JSON fuer einfache Konfigurationsdaten; spaeter optional Godot Resources (.tres) fuer editornahe Daten.
Save	JSON-Dateien unter user:// mit schemaVersion.
Steam	GodotSteam spaeter, aber gekapselt hinter SteamService/PlatformService.
Editor	Godot Editor fuer Szenen; VSCode + Codex fuer Scripts, Daten und Reviews.
Versionskontrolle	Git. Kleine Commits nach jedem sauberen Codex-Task.

5. Gesamtarchitektur

Die Architektur ist in Schichten aufgebaut. Die wichtigste Regel bleibt: Die Simulation entscheidet, die Darstellung zeigt nur an. UI, Karte und Effekte duerfen keine eigenen Spielregeln besitzen.

```
Godot Application
  Main.tscn
    GameRoot
      Managers
        GameManager
        TimeController
        CommandBus
        SaveManager
        AudioManager
        PlatformService
        SteamServiceStub / GodotSteamService later
      World
```

```

WorldMap
  CountryLayer
  ArmyLayer
  EffectsLayer
UI
  Hud
  CountryPanel
  ArmyPanel
  UpgradeModal
  MiniGoalPanel
  TimeControls
  MainMenu
  PauseMenu
Data
  DataRepository

```

Architekturregel

UI ruft Commands auf. Commands veraendern den GameState ueber die Simulation. Visuals reagieren auf State und Events. Kein Button berechnet direkt Kampf, Wirtschaft oder Besitzwechsel.

6. Szenenstruktur in Godot

Godot-Projekte werden ueber Szenen und Nodes organisiert. Damit Codex nicht unkontrolliert Szenen zerlegt, braucht Paper Empire eine sehr stabile Hauptstruktur.

```

res://scenes/
main/
  Main.tscn
  GameRoot.tscn
world/
  WorldMap.tscn
  CountryNode.tscn
  ArmyVisual.tscn
  MissileEffect.tscn
  ExplosionEffect.tscn
ui/
  Hud.tscn
  CountryPanel.tscn
  ArmyPanel.tscn
  UpgradeModal.tscn
  MiniGoalPanel.tscn
  TimeControls.tscn
  MainMenu.tscn
  PauseMenu.tscn
managers/
  GameManager.tscn
  AudioManager.tscn
  SaveManager.tscn

```

Szenen werden im Godot Editor angelegt und grob stabil gehalten. Codex darf Scripts und klar beschriebene Node-Eigenschaften aendern, soll aber nicht ohne explizite Anweisung die komplette Szenenhierarchie umstrukturieren.

7. Autoloads / Singletons

Einige Systeme sollen global erreichbar sein, aber nicht alles wird Singleton. Zu viele Singletons machen das Projekt schwer testbar. Vorschlag:

Bereich	Festlegung
Game	Autoload oder Root-Manager fuer aktuellen Run-State, Commands und GameEvents.

DataRepository	Autoload fuer geladene Laender-, Einheiten-, Upgrade- und Ziel-Daten.
AudioManager	Autoload fuer SFX/Music/UI-Sound und Lautstaerke-Einstellungen.
SaveManager	Autoload fuer Save/Load und MetaProgress.
PlatformService	Autoload als Adapter fuer Desktop/Steam-spezifische Funktionen.

Nicht als Autoload: einzelne UI-Panels, Laender-Nodes, Armee-Visuals und Effekte. Diese gehoeren in ihre Szenen.

8. Kernmodule und Verantwortlichkeiten

8.1 GameManager

- Besitzt oder referenziert den aktuellen GameState.
- Startet neue Runs.
- Nimmt Commands entgegen und leitet sie an die Simulation weiter.
- Sammelt GameEvents fuer UI, Audio und Effekte.
- Kennt keine konkreten UI-Buttons und keine konkreten Partikel-Nodes.

8.2 TimeController

- Verwaltet Pause, 1x, 2x und 4x.
- Erzeugt feste Simulationsticks mit Accumulator.
- Berechnet Ingame-Zeit: Woche, Monat, Jahr.
- Loest Monatsticks fuer Wirtschaft und Unterhalt aus.
- Trennt Simulationszeit von visueller Framerate.

8.3 DataRepository

- Laedt statische Daten aus JSON oder Resource-Dateien.
- Validiert IDs, Nachbarn, Startwerte und Einheitenwerte.
- Stellt nur readonly Daten bereit.
- Keine Spielstandslogik in diesem Modul.

8.4 CommandBus / GameCommands

Commands bilden die Grenze zwischen UI und Simulation. UI-Komponenten rufen Commands auf, statt den State direkt zu veraendern.

```
createArmy(countryId: String) -> void
recruitUnits(countryId: String, unitType: String, amount: int) -> void
moveArmy(armyId: String, targetCountryId: String) -> void
startAttack(armyIds: Array[String], targetCountryId: String) -> void
chooseUpgrade(upgradeId: String) -> void
setGameSpeed(speed: int) -> void
saveGame(slotId: String) -> void
loadGame(slotId: String) -> void
```

8.5 Simulation-Module

- EconomySimulation: Gold, Nahrung, Unterhalt, Monatstick.
- ArmySimulation: Rekrutierung, Bewegung, Statuswechsel.
- CombatSimulation: Kampfkraft, Konterlogik, Verluste, Sieger.
- ConquestSimulation: Besitzwechsel, Upgrade-Auswahl, Belohnung.
- ThreatSimulation: Bedrohung durch Zeit, Kriege, Eroberungen und grosse Armeen.

- GoalSimulation: Mini-Ziele pruefen und Belohnungen bereitstellen.
- UpgradeSimulation: Upgrade-Auswahl rollen und Effekte anwenden.

9. Zeitmodell und Ticks

Paper Empire braucht zwei Update-Ebenen: feste Simulation fuer Regeln und fluessige Visualisierung fuer Animationen.

Bereich	Festlegung
Fester Simulationstick	Eigene Tick-Rate, z. B. 10 oder 20 Simulationsticks pro echter Sekunde. Nicht direkt von FPS abhaengig.
Visuelle Frames	Godot <code>_process(delta)</code> fuer Kamera, Hover, UI-Animationen und visuelle Interpolation.
Physics Process	<code>_physics_process(delta)</code> kann fuer feste Updates genutzt werden. Da Paper Empire kein Physikspiel ist, bleibt ein eigener Simulations-Accumulator trotzdem die zentrale Regel.
Monatstick	Wird aus Ingame-Zeit abgeleitet. Beispiel: 1 Ingame-Woche = 5 Sekunden bei 1x; 4 Wochen = 1 Monat.
Armee-Bewegung	Der echte Zustand bewegt sich im Simulationstick; das sichtbare Icon interpoliert weich zwischen Start/Ziel.
Pause	Simulationsticks stoppen. UI bleibt bedienbar. Menues und Hover duerfen weiter funktionieren.

```

Visual frame:
    _process(delta)
        updateCamera(delta)
        updateUiAnimations(delta)
        interpolateArmyVisuals(delta)
        updateEffects(delta)

Simulation:
    accumulatedTime += delta * gameSpeed
    while accumulatedTime >= fixedStep:
        advanceSimulation(fixedStep)
        accumulatedTime -= fixedStep

Month tick:
    if ingameWeeksSinceLastMonth >= 4:
        applyMonthlyEconomy()
        applyArmyUpkeep()
        checkFoodShortage()

```

Warum nicht alles in `_process`?

Wenn Kampf, Einkommen und Bewegungen direkt pro Frame berechnet werden, veraendert sich das Spielverhalten je nach FPS. Paper Empire muss fuer Debugging und Balancing moeglichst deterministisch bleiben.

10. GameState und Datenmodell

Der GameState ist der aktuelle Run. Er muss serialisierbar bleiben, damit Save/Load, Debugging und spaeter Steam Cloud einfacher werden. Godot-Nodes duerfen nicht direkt Teil des GameState sein.

10.1 GameState

```

class_name GameState

var runId: String
var playerId: String

```

```

var runStatus: String # menu, running, paused, won, lost
var speed: int # 0, 1, 2, 4
var time: gameTime
var resources: PlayerResources
var countries: Dictionary # countryId -> CountryState
var armies: Dictionary # armyId -> ArmyState
var activeBattles: Dictionary
var activeUpgradeChoice: UpgradeChoice
var miniGoals: Array[MiniGoalState]
var eventQueue: Array[GameEvent]

```

10.2 CountryData vs CountryState

Statische und dynamische Daten werden getrennt.

Bereich	Festlegung
CountryData	Name, Basisgold, Basisnahrung, Basisverteidigung, Nachbarn, Kartenform, Zentrum, Tags.
CountryState	Besitzer, stationierte Armeen, temporäre Modifikatoren, Beschädigung, aktueller Verteidigungsstatus.

```

CountryData:
  id: "germany"
  displayName: "Deutschland"
  goldPerMonth: 150
  foodPerMonth: 20
  baseDefense: 620
  neighborCountryIds: ["france", "poland", "austria", "czechia"]
  center: { "x": 640, "y": 380 }

```

```

CountryState:
  countryId: "germany"
  ownerId: "player"
  armyIds: ["army_001"]
  defenseModifier: 1.0
  isThreatened: false

```

10.3 ArmyState

```

ArmyState:
  id: "army_001"
  ownerId: "player"
  locationCountryId: "germany"
  targetCountryId: "poland"
  status: "moving" # idle, moving, fighting, retreating
  movementProgress: 0.35
  movementDurationSeconds: 18.0
  units:
    infantry: 6000
    cavalry: 2000
    artillery: 800

```

10.4 UpgradeData

```

UpgradeData:
  id: "efficientSupply_1"
  displayName: "Effiziente Versorgung"
  rarity: "common"
  description: "-10 Prozent Nahrungskosten fuer Armeen."
  effectType: "armyFoodCostMultiplier"
  value: 0.90

```

11. Weltkarte

Die Weltkarte ist das groesste technische Risiko beim Wechsel zu Godot, weil Web/SVG hier sehr bequem gewesen waere. Deshalb wird die Karte bewusst in Stufen aufgebaut.

- Erst eine Testkarte mit 10-20 Laendern als manuell gezeichnete Polygone.
- Danach Europa/Teilwelt mit besseren Laenderformen.
- Erst spaeter komplette Weltkarte mit allen groesseren Laendern.

11.1 CountryNode

```
CountryNode.tscn
Area2D (CountryNode.gd)
    Polygon2D          # sichtbare Flaeche
    CollisionPolygon2D  # Klick- und Hover-Flaeche
    Label              # optionaler Landesname im Debug
    Marker2D CenterPoint # Zentrum fuer Armeen, Raketen, Icons
```

- Area2D empfaengt Hover und Klicks.
- Polygon2D zeigt Besitzerfarbe, Hover-Farbe und Auswahlrand.
- CollisionPolygon2D muss grob zur Landesform passen, darf fuer MVP vereinfacht sein.
- CenterPoint wird fuer Armee-Icons, Pfeile und Raketen verwendet.

11.2 Map Coordinates

Alle Laenderzentren, Armeewege und Effekte laufen im Koordinatensystem der WorldMap. Die Kamera skaliert und verschiebt nur die Darstellung. GameState speichert keine Screen-Koordinaten, sondern Country-IDs und Fortschrittswerte.

11.3 Adjazenz / Nachbarn

Nachbarn werden nicht aus Polygonen berechnet. Fuer Gameplay ist eine explizite Nachbarliste robuster und leichter zu balancen. Das erlaubt spaeter auch Sonderregeln, z. B. Seeuebergaenge oder gesperrte Grenzen.

```
countryData.neighborCountryIds = ["france", "poland", "austria"]

isMoveAllowed(fromCountryId, toCountryId):
    return toCountryId in DataRepository.getCountry(fromCountryId).neighborCountryIds
```

12. Armeen und Bewegung

Armeen sind Spielzustand plus Visualisierung. Der echte Zustand liegt in ArmyState. Das sichtbare ArmyVisual ist nur eine Darstellung davon.

Bereich	Festlegung
ArmyState	Daten: Besitzer, Standort, Ziel, Einheiten, Status, Bewegungsfortschritt.
ArmyVisual	Node2D mit Icon, Zahl, kleinen Bannern und Bewegungstween/Interpolation.
ArmyLayer	Erstellt, aktualisiert und entfernt ArmyVisuals basierend auf GameState.
Movement	Simulation erhoehrt movementProgress; Visual interpoliert zwischen Country-Centern.
Combat Trigger	Wenn movementProgress >= 1 und Ziel feindlich ist, startet Kampf oder Armee wird stationiert.

Eine Armee bewegt sich symbolisch auf der Karte. Es ist kein RTS mit genauer Pfadfindung einzelner Soldaten. Fuer den MVP reicht Bewegung zwischen Laenderzentren oder ueber einfache Bezier-Kurven.

13. Kampf und Eroberung

Kaempfe laufen automatisch. Spielerentscheidungen finden vorher statt: Armee zusammenstellen, Ziel waehlen, Truppen positionieren, Angriff starten.

Bereich	Festlegung
calculateCombatPower	Berechnet Kampfkraft aus Einheitentypen, Upgrades, Nahrung, Verteidigung und ggf. Konterboni.
resolveBattleTick	Reduziert Kampfstaerke/Verluste ueber Zeit oder berechnet Kampf in diskreten Schritten.
finishBattle	Bestimmt Sieger, Verluste, Eroberung, Events und Upgrade-Auswahl.
conquerCountry	Setzt Besitzer, transferiert Land, erzeugt countryConquered Event.

Schere-Stein-Papier der MVP-Einheiten

Infanterie haelt und ist guenstig, Kavallerie ist schnell und gut gegen Artillerie, Artillerie ist stark gegen defensive Infanterie/Laenderverteidigung. Die genauen Zahlen gehoeren ins Balancing, nicht fest in UI oder CountryNode.

14. Effekte und Symbolik

Godot uebernimmt die Effekte direkt in der Engine. Raketen, Explosionen und Einschlaege sind Visualisierung von GameEvents, keine eigene Spiellogik.

```
GameEvent: missileLaunched
  fromCountryId: "germany"
  targetCountryId: "france"
  effectId: "nuke_basic"

EffectsLayer:
  start = WorldMap.getCountryCenter(fromCountryId)
  target = WorldMap.getCountryCenter(targetCountryId)
  spawn MissileEffect
  animate missile along curve
  on impact:
    spawn ExplosionEffect
    play sound
    emit visual event completed
```

- MissileEffect: Node2D mit Sprite/Line2D, Tween entlang einer Kurve, optional Trail.
- ExplosionEffect: Partikel, Flash-Kreis, Kamerashake, Sound.
- BattleEffect: pulsierendes Land, kleine Kampfmarker, Rauch oder Schwerter-Icon.
- ConquestEffect: Flaggenwechsel, kurzer Kartenflash, UI-Event.

Effekte duerfen niemals Besitz, Ressourcen oder Kampfausgang selbst aendern. Sie reagieren nur auf Events, die aus der Simulation kommen.

15. UI-Architektur

Paper Empire bleibt ein UI-lastiges Strategiespiel. Deshalb muss die Godot-UI bewusst strukturiert werden und darf nicht in wild platzierten Einzelbuttons enden.

```
UIRoot (CanvasLayer)
  Hud
    TopResourceBar
    MiniGoalPanel
    TimeControls
  Panels
    PlayerPanel
```

- CountryPanel
- ArmyPanel
- ShopPanel
- Modals
 - UpgradeModal
 - ConfirmDialog
 - PauseMenu
- Tooltips
 - TooltipLayer

- UI liest State ueber GameManager oder ViewModels.
- UI sendet nur Commands.
- UI zeigt keine eigene Kampf- oder Wirtschaftslogik.
- Komplexe Panels bekommen eigene Scripts und klare Input/Output-Methoden.
- 1920x1080 ist die erste Zielaufloesung. UI-Scale wird spaeter ergaenzt.

16. Audio

Bereich	Festlegung
Audio-Busse	Master, Music, SFX, UI.
AudioManager	Zentraler Zugriff fuer playSfx, playMusic, setVolume, mute.
2D-Sound	Nur falls Positionsaudio wirklich nuetzlich ist. Fuer UI/Strategiekarte reichen meist normale SFX.
Events	Audio reagiert auf GameEvents: battleStarted, countryConquered, missileLaunched, explosionImpact.

17. Save/Load und Meta-Fortschritt

Der Save-Stand muss unabhaengig von Godot-Nodes sein. Gespeichert werden Daten, nicht Szenen.

Bereich	Festlegung
Run Save	Aktueller Run: Laenderbesitz, Armeen, Ressourcen, Upgrades, Bedrohung, Zeit, aktive Ziele.
Meta Save	Kronen, Shop-Upgrades, laenderspezifische Boni, Einstellungen.
Pfad	user://paper_empire/save_slot_1.json und user://paper_empire/meta.json.
Versionierung	Jede Save-Datei hat schemaVersion. Spaetere Migrationen werden dadurch moeglich.
Steam Cloud	Spaeter ueber Steam-Konfiguration oder Remote Storage. Im Code bleibt SaveManager die zentrale Schnittstelle.

```
Save file root:
{
  "schemaVersion": 1,
  "gameVersion": "0.1.0",
  "createdAt": "2026-05-09T12:00:00Z",
  "runState": { ... },
  "metaState": { ... }
}
```

18. Steam-Strategie

Steam wird vorbereitet, aber nicht in den ersten technischen Prototyp eingebaut. Der Code muss auch ohne laufenden Steam-Client funktionieren. Steam-spezifische Funktionen laufen hinter einem Adapter.

```

PlatformService
  isSteamAvailable() -> bool
  unlockAchievement(id: String) -> void
  setStat(id: String, value: int) -> void
  storeSaveToCloud(path: String) -> void

Implementations:
  DesktopPlatformService # no Steam, local only
  GodotSteamPlatformService # later

```

- MVP: keine Steam-Abhängigkeit.
- Vertical Slice: lokale Desktop-Builds testen.
- Pre-Steam: AppID, Steamworks-Setup, Depot-Struktur, Store Assets.
- Später: Achievements, Stats, Cloud Saves und Overlay testen.
- SteamPipe/Depots sind unabhängig von der Engine und kommen erst bei Distribution dazu.

19. Projektstruktur

```

res://
  addons/
    godotsteam/                # later, not in MVP
  assets/
    art/
    audio/
    fonts/
    map/
  data/
    countries.json
    units.json
    upgrades.json
    miniGoals.json
    startCountries.json
  scenes/
    main/
    managers/
    world/
    ui/
    effects/
  scripts/
    core/
      game_state.gd
      game_time.gd
      game_event.gd
    simulation/
      economy_simulation.gd
      army_simulation.gd
      combat_simulation.gd
      conquest_simulation.gd
      threat_simulation.gd
      upgrade_simulation.gd
      goal_simulation.gd
    commands/
      command_bus.gd
      game_commands.gd
    managers/
      game_manager.gd
      time_controller.gd
      save_manager.gd
      audio_manager.gd
      data_repository.gd
      platform_service.gd
    world/
      world_map.gd

```

```

country_node.gd
army_layer.gd
army_visual.gd
ui/
  hud.gd
  country_panel.gd
  army_panel.gd
  upgrade_modal.gd
effects/
  effects_layer.gd
  missile_effect.gd
  explosion_effect.gd
saves/                                # not committed; runtime uses user://
docs/
  architecture.md
  codex_rules.md

```

20. Codex-Regeln fuer Godot

Codex kann bei Godot helfen, aber es braucht klare Grenzen. Es soll nicht wie bei einem reinen TypeScript-Projekt blind grosse Teile umbauen.

- Codex arbeitet in VSCode am Projektordner.
- Godot Editor bleibt die Quelle fuer visuelle Szenenbearbeitung.
- Codex darf GDScript-Dateien, JSON-Daten, Markdown-Dokumente und kleine klar definierte .tscn-Aenderungen bearbeiten.
- Codex darf keine Szenenhierarchie grossflaechig umstrukturieren, ausser der Task verlangt es explizit.
- Nach jedem Codex-Task Godot starten und die Hauptszene testen.
- Nach jedem funktionierenden Task committen.

20.1 AGENTS.md fuer Codex

```

# Paper Empire - Codex Rules

## Project Goal
Paper Empire is a desktop-first Steam strategy roguelike built with Godot 4 and GDScript.
There is no web version in the main plan.

## Core Rules
- Use GDScript for gameplay code.
- Use English names and comments in code.
- Use lowerCamelCase for custom project identifiers where this does not conflict with Godot APIs.
- Godot lifecycle methods keep their required names, e.g. _ready, _process, _physics_process.
- Keep simulation logic out of UI scripts.
- UI scripts may display state and call commands, but must not calculate combat, economy, movement or threat.
- Effects react to GameEvents. Effects must not change gameplay state.
- Keep Godot Nodes out of serializable save data.
- Do not add Steam/GodotSteam unless the task explicitly asks for it.
- Keep changes small and focused.

## Scene Safety
- Do not rewrite large .tscn files unless explicitly requested.
- Prefer adding scripts and data files over restructuring scenes.
- If a Node path is required, document it in the script header.

## Checks
- Open the project in Godot after changes.
- Run the main scene.
- Verify there are no parser errors.
- Verify the task-specific behavior manually.

```

```
## Architecture Boundaries
- GameManager owns run state and command execution.
- DataRepository owns static data loading.
- TimeController owns speed and fixed simulation ticks.
- WorldMap owns country nodes and map interaction.
- UI owns panels and presentation only.
- EffectsLayer owns missiles, explosions and other visual effects only.
```

20.2 Beispiel-Prompt fuer Codex

Implement only the requested task.

Context:

Paper Empire is a Godot 4 desktop strategy game. Follow AGENTS.md.

Task:

Create scripts/core/game_time.gd and scripts/managers/time_controller.gd.

The TimeController should support pause, 1x, 2x and 4x speed and expose a fixed simulation tick signal.

Do not implement UI.

Do not implement economy.

Do not modify existing scenes unless absolutely necessary.

Acceptance criteria:

- Code compiles in Godot.
- TimeController has typed properties.
- There is a signal for fixed simulation ticks.
- There is a signal for month ticks.
- No gameplay logic outside TimeController is added.

21. Coding Standards

- Code comments and identifiers in English.
- Custom project identifiers use lowerCamelCase where reasonable.
- Required Godot callback names remain as Godot expects: _ready, _process, _physics_process, _input, etc.
- Prefer typed GDScript: var gold: int, func startRun(startCountryId: String) -> void.
- Avoid clever abstractions until repeated pain proves they are needed.
- Keep files small. Split large managers when responsibilities become unclear.
- Do not reference UI Nodes from simulation modules.
- Do not store Node references in GameState or save data.
- Every ID in JSON must be stable and lowercase, e.g. germany, infantry, efficientSupply1.

22. Testing und Debugging

Godot hat nicht denselben Standard-Testworkflow wie ein TypeScript-Projekt. Fuer Paper Empire wird deshalb pragmatisch gestartet: reine Logik wird klein gehalten und ueber Debug-Szenen, Test-Commands und spaeter optional ein Godot-Testframework geprueft.

Bereich	Festlegung
Debug Scene	Eine Szene DebugSandbox.tscn fuer isoliertes Testen von Karte, Armee, Kampf und Effekten.
Debug Commands	Tasten oder Debug-Menue fuer Gold geben, Land erobern, Rakete starten, Upgrade rollen.
Assertions	Datenvalidierung beim Start: fehlende Country-IDs, kaputte Nachbarn, ungueltige Units.
Smoke Test	Projekt starten, neuer Run, Land anklicken, Armee erstellen,

	Angriff starten, Eroberung abschliessen.
Optional spaeter	GdUnit4 oder aehnliches Testframework, falls Logiktests regelmassig gebraucht werden.

23. Build, Export und Distribution

- Zuerst nur Godot Editor Play-Mode.
- Dann lokaler Windows-Debug-Export.
- Danach Windows-Release-Export.
- Erst nach Vertical Slice: Steamworks-Account, AppID, Depot-Struktur, Store-Seite und SteamPipe-Builds.
- Linux und macOS erst nach stabilem Windows-Build pruefen.

Steam nicht zu frueh einbauen

Steamworks-Integration erzeugt zusaetzliche Fehlerquellen. Fuer den ersten spielbaren Prototyp ist ein sauberer lokaler Windows-Build wichtiger als Achievements, Cloud Saves oder Overlay.

24. Risiken und Gegenmassnahmen

Risiko	Warum relevant	Gegenmassnahme
Weltkarte in Godot ist aufwaendiger als SVG	Laenderformen, Klickflaechen und Nachbarn sind nicht automatisch vorhanden.	MVP mit 10-20 Laendern; Import-Pipeline erst spaeter; Nachbarn manuell pflegen.
Codex kann Szenen beschadigen	.tscn-Dateien sind textbasiert, aber empfindlich gegen falsche Node-Pfade.	AGENTS.md; kleine Tasks; Szenenstruktur stabil halten; Git-Commits nach jedem Schritt.
UI wird unuebersichtlich	Strategiespiel braucht viele Panels.	UIRoot mit klaren Panels; Commands statt direkter State-Aenderungen; 1920x1080 zuerst.
Simulation wird framerate-abhaengig	Balancing und Debugging werden sonst unzuverlaessig.	Eigener fester Simulationstick und visuelle Interpolation.
Steam zu frueh macht MVP langsam	GodotSteam, AppID und Steam-Callbacks koennen ablenken.	PlatformService-Stub zuerst, Steam erst nach Vertical Slice.
Scope creep	Grand Strategy kann schnell explodieren.	MVP bleibt bei Laendern, Gold, Nahrung, Bedrohung, Armeen, Upgrades, Mini-Zielen.

25. Aktualisierte Architektur-Entscheidung

Paper Empire wird als Godot-4-Desktop-Spiel gebaut. Steam ist das spaetere Ziel, wird aber technisch entkoppelt vorbereitet. Die Simulation ist datengetrieben, serialisierbar und von UI/Visuals getrennt. Godot uebernimmt 2D-Darstellung, UI, Animationen, Partikel, Audio und Desktop-Export.

Finale Stack-Festlegung

Godot 4 + GDScript + Godot Control UI + 2D-WorldMap + Godot Effects + lokale Saves + spaeter GodotSteam. Keine Webversion, kein Electron, kein React.

26. Was als Naechstes kommt

Der naechste separate Schritt ist ein Godot-spezifischer Codex-Schrittplan. Dieser sollte nicht direkt das ganze Spiel bauen lassen, sondern zuerst Projektanlage, AGENTS.md, Basis-Szenen, GameState, TimeController, DataRepository und Testkarte einzeln definieren.

Der alte Electron-Schrittplan darf nicht weiter als Arbeitsgrundlage verwendet werden. Er kann nur noch als inhaltliche Referenz fuer Modulgrenzen dienen.

27. Technische Referenzen

- Godot Docs - Idle and Physics Processing:
https://docs.godotengine.org/en/stable/tutorials/scripting/idle_and_physics_processing.html
- Godot Docs - Exporting for Windows:
https://docs.godotengine.org/en/4.4/tutorials/export/exporting_for_windows.html
- Godot Asset Library - GodotSteam GDExtension: <https://godotengine.org/asset-library/asset/2445>
- Steamworks Docs - Uploading to Steam / SteamPipe: <https://partner.steamgames.com/doc/sdk/uploading>
- Steamworks Docs - Depots: <https://partner.steamgames.com/doc/store/application/depots>

Dokumentstatus: Architektur- und Techplan. Kein detaillierter Implementierungs-Schrittplan. Dieser folgt separat.